Laborator 04

The english translation

Problema 1:

Scrieti o functie care primeste ca parametru un String regex, un String text si un int x, si returneaza substringurile de lungime x sau mai mari care fac match pe expresia regulata.

Problema 2:

Implementati o Stiva folosind fisiere ca storage. (pop, push, top)

Problema 3:

Implementati clasa MathOps unde T si G pot fi int, double sau String:

```
MathOps <T,G> {
    dynamic sub(T obj1, G obj2);
    dynamic mul(T obj1, G obj2);
}
```

Pentru String implementati sub ca diferenta dintre lungimile celor doua stringuri si mul va arunca exceptie.

Problema 4:

Implementanti Map<String,dynamic> jsonSumJson(String jsonPath1, String jsonPath2), unde jsonPath1 si jsonPath2 sunt path-uri catre doua fisiere json. Operatia de adunare intre 2 fisiere json ar trebui sa rezulte intr-un json in care pentru nodurile comune se va alege valoarea din cel de-al doilea fisier json. Functia va fi scrisa intr-o librarie separata de cea din care va fi apelata.

English

Exercise 1:

Write a function that takes as parameters a String regex, a String text, and an int x, and returns the substrings of length x or larger, that match the regular expression.

Exercise 2:

Implement a Stack using files as storage. (pop, push, top)

Exercise 3:

Implement the MathOps class where T and G can be int, double, or String.

```
MathOps <T,G> {
    int sub(T obj1, G obj2);
    int mul(T obj1, G obj2);
}
```

In the case of String, sub should return the difference between the lengths of the two strings and mul should throw an exception.

Exercise 4:

Implement a function Map<String, dynamic> jsonSumJson(String jsonPath1, String jsonPath2), where jsonPath1 and jsonPath2 are paths to two JSON files. The operation of summing two JSON files should result in a JSON where, for common nodes, the value from the second JSON file will be chosen. The function will be written in a separate library from the one where it will be called.